

Next.js and Tailwind CSS Full-Stack Comparison Guide

Basic to Advanced | Architecture, Caching, Server Actions, Styling
Decision Guide, Capstone Project and Interview Prep

Contents

How to Use This Guide	5
The 3-volume series	5
Master category map: what each technology IS	5
Versions this guide targets	6
Study plan	6
 1. Next.js - The Big Picture	 7
1.1 What problem does Next.js solve?	7
1.2 Create a project	7
1.3 Project structure (App Router)	7
1.4 Special files	8
1.5 App Router vs Pages Router	8
1.6 Root layout	9
 2. Next.js - Routing and Basics	 10
2.1 Pages and nested layouts	10
2.2 Route types	10
2.3 Dynamic routes and search params	10
2.4 Navigation	11
2.4.1 Loading and error UI	12
2.5 Metadata and SEO	12
2.6 Images, fonts, scripts	12
2.7 Environment variables	13
2.8 Configuration	13
 3. Server and Client Components	 14
3.1 The two kinds of components	14
3.2 The "use client" boundary	14
3.3 Composition rules	15
3.4 Context providers	15
3.5 Server-only and client-only code	16
3.6 When to choose which - quick guide	16
 4. Next.js - Data Fetching, Rendering and Caching	 17
4.1 Fetching data in a Server Component	17
4.2 Sequential vs parallel fetching	17
4.3 Streaming with Suspense	17
4.4 Static vs dynamic rendering	18
4.5 Caching model in Next.js 16 (Cache Components)	18
4.6 Request-time APIs	19

4.7 Client-side fetching is still useful for...	20
5. Next.js - Mutations, Server Actions and Route Handlers	21
5.1 Server Actions	21
5.2 Server Actions vs Route Handlers	22
5.3 Route Handlers	22
5.4 proxy.ts (formerly middleware.ts)	23
5.5 Comparison: same endpoint in three stacks	24
6. Next.js - Advanced	25
6.1 Authentication and authorisation	25
6.2 Parallel and intercepting routes (modals with URLs)	25
6.3 Runtimes	26
6.4 Performance checklist	26
6.5 Security checklist	26
6.6 Testing strategy	27
6.7 Deployment options	27
6.8 Next.js as a Backend-for-Frontend (BFF) for Spring Boot	28
6.9 Other Next.js features worth knowing	28
7. Tailwind CSS - Basics	29
7.1 The idea	29
7.2 Install (Tailwind v4)	29
7.3 Core utilities by category	30
7.4 Layout: Flexbox	31
7.5 Layout: Grid	31
7.6 Display, position and overflow	31
7.7 Responsive design (mobile first)	32
7.8 State variants	32
7.9 Dark mode	33
7.10 A complete component	33
8. Tailwind CSS - Intermediate and Advanced	34
8.1 Customising the theme with @theme (v4)	34
8.2 v3 vs v4 (migration cheat sheet)	35
8.3 Reusable components: the right way	35
8.4 @apply, @layer, @utility	36
8.5 Arbitrary values and modern CSS features	36
8.6 Animation and transitions	36
8.7 Accessibility with Tailwind	37
8.8 Component libraries on Tailwind	37
8.9 Performance and tooling	37

8.10 Tailwind common mistakes	38
---	----

9. Deep Comparisons 39

9.1 Next.js vs other React / full-stack frameworks	39
9.2 Tailwind CSS vs other styling approaches	39
9.3 Full-stack architecture options compared	40
9.4 The whole technology comparison on one page	41
9.5 Choosing by project type	41

10. Capstone: Full-Stack Task Manager (Next.js + Tailwind + TypeScript + Spring Boot) 43

10.1 Architecture	43
10.2 Folder layout	43
10.3 Typed API client and schemas	43
10.4 List page (Server Component) with Tailwind	44
10.5 Server Actions	45
10.6 Client Component with optimistic update	45
10.7 What the capstone proves	46

11. Decision Guide, Interview Q&A and Cheat Sheets 47

11.1 Next.js interview questions	47
11.2 Tailwind interview questions	47
11.3 Mixed "which one" questions	48
11.4 Syntax cheat sheet: Next.js	48
11.5 Syntax cheat sheet: Tailwind	49
11.6 Master roadmap across all three volumes	49
11.7 Final one-page summary	49

How to Use This Guide

CATEGORY

Volume 3 of 3 - Full-stack Framework and Styling. Covers Next.js (the React framework) and Tailwind CSS (utility-first styling) from beginner to advanced, then ties all three volumes together with architecture comparisons and a decision guide. Level: Beginner to Advanced.

The 3-volume series

Volume	Topics	Category
Vol 1 - Backend	Spring Boot, Node.js, Express, NestJS	Server-side / API
Vol 2 - Language and UI	TypeScript, React	Language + client-side UI
Vol 3 - Full-stack and Styling (this file)	Next.js, Tailwind CSS, architecture and decision guide	Meta-framework + styling

Master category map: what each technology IS

Confusion usually starts because these technologies belong to **different categories**. They are not all competitors; many are used together.

Technology	Category	One-line description	Competes with
Java	Programming language	Typed, compiled language for the JVM	Kotlin, C#, Go
Spring Boot	Backend framework	Opinionated Java framework for APIs and services	Quarkus, Micronaut, NestJS, ASP.NET
Node.js	JavaScript runtime	Runs JavaScript on the server (V8 + event loop)	Deno, Bun
Express / Fastify	Backend web library	Minimal routing + middleware on Node	Koa, Hono
NestJS	Backend framework	Structured Node framework (Angular/Spring style)	Spring Boot (in style)
TypeScript	Programming language (superset of JS)	JavaScript plus static types	Flow, plain JS
React	UI library	Component-based UI (client and server components)	Vue, Angular, Svelte, Solid
Next.js	Full-stack React meta-framework	React + routing + SSR/SSG + server actions + API routes	Remix / React Router, Nuxt, SvelteKit, Astro
Tailwind CSS	CSS framework (utility-first)	Style by composing small utility classes	Bootstrap, CSS Modules, styled-components
Vite	Build tool	Fast dev server and bundler	Webpack, Turbopack, Parcel

TEXT

YOUR APPLICATION

STYLING	Tailwind CSS	
UI FRAMEWORK	Next.js (routing, rendering, server actions)	<- Volume 3
UI LIBRARY	React (components, hooks)	<- Volume 2
LANGUAGE	TypeScript (types over JavaScript)	<- Volume 2
RUNTIME	Node.js (runs Next.js and JS on the server)	<- Volume 1
BACKEND API	Spring Boot (Java) or Node/Express/NestJS	<- Volume 1
DATABASE	PostgreSQL / MySQL / MongoDB / Redis	

Versions this guide targets

Technology	Version	Key facts
Next.js	16.x (App Router)	Turbopack default bundler, Cache Components (" use cache "), proxy.ts replaces middleware.ts , React 19.2, async params
React	19.2	Server Components, Actions, Compiler (opt-in, stable)
Tailwind CSS	4.x	CSS-first config (@import "tailwindcss" , @theme), Rust engine (Oxide), no tailwind.config.js required
Node.js	20.9+ (22 / 24 LTS recommended)	Required by Next.js 16
TypeScript	5.x	Next.js generates types and supports next.config.ts

NOTE

The Next.js team changes caching and file conventions between majors. Where syntax differs between Next.js 15 and 16, the difference is noted. Always confirm on nextjs.org/docs for your installed version.

Study plan

1. **Week 1:** Next.js basics: routing, layouts, links, metadata (Ch 1-3).
2. **Week 2:** Data, caching, Server Actions, route handlers (Ch 4-5).
3. **Week 3:** Tailwind fundamentals and component patterns (Ch 7-8).
4. **Week 4:** Advanced Next.js, architecture comparison, full example (Ch 6, 9-10), then revision (Ch 11-12).

1. Next.js - The Big Picture

CATEGORY

Framework: full-stack React. What Next.js adds on top of React, the App Router mental model, project creation and folder structure. Used for marketing sites, e-commerce, SaaS dashboards, blogs and full-stack apps. Level: Beginner.

1.1 What problem does Next.js solve?

Plain React only gives you the UI layer. For a real product you also need routing, data loading, server rendering (SEO and speed), image and font optimisation, API endpoints, bundling, caching and deployment. **Next.js bundles all of these** with sensible defaults.

Need	Plain React (Vite SPA)	Next.js
Routing	Install React Router and configure	File-system routing (folders are URLs)
First page load / SEO	Empty HTML, JS renders later	HTML rendered on the server or at build time
Data fetching	<code>useEffect</code> / TanStack Query in the browser	<code>await</code> directly in Server Components
Backend endpoints	Separate server needed	Route Handlers and Server Actions in the same project
Code splitting	Manual lazy imports	Automatic per route
Image / font optimisation	Manual	<code>next/image</code> , <code>next/font</code>
Caching and revalidation	Manual	Built in (" <code>use cache</code> ", tags, revalidate)
Deploy	Static hosting	Node server, serverless, Docker, Vercel

1.2 Create a project

BASH

```
npx create-next-app@latest my-app
# Prompts: TypeScript? Yes | ESLint? Yes | Tailwind CSS? Yes | src/ directory? (your choice)
#           App Router? Yes | React Compiler? optional | import alias @/* ? Yes
cd my-app
npm run dev      # http://localhost:3000 (Turbopack)
npm run build    # production build
npm run start    # run the production build
```

1.3 Project structure (App Router)

TEXT

```

my-app/
  app/
    layout.tsx      <- the router: folders = URL segments
    page.tsx        <- root layout (required): <html> and <body>
    globals.css     <- route "/"
    loading.tsx     <- Tailwind import + global styles
    error.tsx       <- instant loading UI (Suspense) for this segment
    not-found.tsx   <- error boundary UI
    tasks/
      page.tsx      <- 404 UI
      [id]/page.tsx <- route "/tasks"
      actions.ts    <- route "/tasks/42" (dynamic segment)
    api/
      health/route.ts <- Server Actions ("use server")
    components/     <- Route Handler: GET /api/health
    lib/            <- shared components (not routes)
    public/         <- db client, api helpers, utils
    proxy.ts        <- static files served from "/"
    next.config.ts  <- request interception (was middleware.ts before v16)
    tsconfig.json   <- configuration
    package.json

```

Only folders containing a **special file** (`page.tsx`, `route.ts`) become public URLs. Other files in the folder (components, helpers) are safe to colocate.

1.4 Special files

File	Purpose
<code>layout.tsx</code>	Shared UI that wraps child routes; preserved across navigation (state is not reset)
<code>page.tsx</code>	The unique UI of a route; makes the segment publicly accessible
<code>template.tsx</code>	Like a layout but remounts on every navigation
<code>loading.tsx</code>	Shown instantly while the segment streams in (wraps the page in <code><Suspense></code>)
<code>error.tsx</code>	Error boundary for the segment (must be a Client Component)
<code>global-error.tsx</code>	Root-level error boundary
<code>not-found.tsx</code>	UI for <code>notFound()</code> and unmatched URLs
<code>route.ts</code>	HTTP endpoint (Route Handler)
<code>default.tsx</code>	Fallback for parallel routes
<code>proxy.ts</code>	Runs before a request completes (redirects, rewrites, headers, auth checks)
<code>instrumentation.ts</code>	Startup hook for observability (OpenTelemetry)

1.5 App Router vs Pages Router

Aspect	Pages Router (<code>pages/</code>)	App Router (<code>app/</code>)
Status	Legacy, still supported	Recommended default
Components	All client components (hydrated)	Server Components by default
Data fetching	<code>getServerSideProps</code> , <code>getStaticProps</code>	<code>async</code> components, <code>fetch</code> , <code>"use cache"</code>
Layouts	<code>_app.tsx</code> / custom	Nested <code>layout.tsx</code>
Streaming and Suspense	Limited	First class
API	<code>pages/api/*</code>	<code>route.ts</code> and Server Actions
React features	React 18 style	React 19: Actions, <code>use</code> , Server Components

1.6 Root layout

TSX

```
// app/layout.tsx
import type { Metadata } from "next";
import { Inter } from "next/font/google";
import "./globals.css";

const inter = Inter({ subsets: ["latin"], display: "swap" }); // self-hosted, no layout shift

export const metadata: Metadata = {
  title: { default: "TaskFlow", template: "%s | TaskFlow" },
  description: "Simple task management",
};

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en" className={inter.className}>
      <body className="min-h-screen bg-white text-gray-900 antialiased">
        <header className="border-b p-4 font-semibold">TaskFlow</header>
        <main className="mx-auto max-w-3xl p-4">{children}</main>
      </body>
    </html>
  );
}
```

2. Next.js - Routing and Basics

CATEGORY

Framework: file-based routing. Pages, layouts, dynamic routes, navigation, metadata, images, fonts, environment variables and styling. Level: Beginner.

2.1 Pages and nested layouts

TSX

```
// app/page.tsx -> "/"
export default function Home() {
  return <h1 className="text-3xl font-bold">Welcome</h1>;
}

// app/dashboard/layout.tsx -> wraps /dashboard and everything below it
export default function DashboardLayout({ children }: { children: React.ReactNode }) {
  return (
    <div className="grid grid-cols-[14rem_1fr]">
      <aside className="border-r p-4">Sidebar</aside>
      <section className="p-6">{children}</section>
    </div>
  );
}

// app/dashboard/page.tsx -> "/dashboard"
// app/dashboard/settings/page.tsx -> "/dashboard/settings"
```

2.2 Route types

Folder name	URL	Notes
app/about/page.tsx	/about	Static segment
app/tasks/[id]/page.tsx	/tasks/42	Dynamic segment, <code>params.id</code>
app/docs/[...slug]/page.tsx	/docs/a/b/c	Catch-all, <code>slug: string[]</code>
app/shop/[...slug]/page.tsx	/shop and /shop/a/b	Optional catch-all
app/(marketing)/pricing/page.tsx	/pricing	Route group: parentheses organise code without affecting the URL
app/_components/	not routable	Private folder (underscore prefix)
app/@modal/...	slot	Parallel routes (render several pages in one layout)
app/(.)photo/[id]/page.tsx	intercepts /photo/[id]	Intercepting routes (modal over current page)

2.3 Dynamic routes and search params

In Next.js 15 and 16, `params` and `searchParams` are **Promises** and must be awaited.

TSX

```
// app/tasks/[id]/page.tsx
import { notFound } from "next/navigation";

type Props = {
  params: Promise<{ id: string }>;
  searchParams: Promise<{ tab?: string }>;
};
export default async function TaskPage({ params, searchParams }: Props) {
  const { id } = await params; // "42"
  const { tab = "details" } = await searchParams; // ?tab=comments

  const res = await fetch(`${process.env.API_URL}/api/tasks/${id}`);
  if (res.status === 404) notFound(); // renders not-found.tsx
  const task: Task = await res.json();
  return (
    <article>
      <h1 className="text-2xl font-bold">{task.title}</h1>
      <p className="text-sm text-gray-500">Tab: {tab}</p>
    </article>
  );
}
```

2.4 Navigation

TSX

```
import Link from "next/link";

<Link href="/tasks">Tasks</Link> // client-side navigation + automatic prefetch
<Link href={`/${tasks.task.id}`} prefetch={false}>Open</Link>
<Link href={{ pathname: "/tasks", query: { status: "OPEN" } }}>Open tasks</Link>
```

TSX

```
"use client";
import { useRouter, usePathname, useSearchParams, useParams } from "next/navigation";

export function Filters() {
  const router = useRouter();
  const pathname = usePathname(); // "/tasks"
  const search = useSearchParams(); // read ?status=OPEN
  const params = useParams(); // { id: "42" } for dynamic routes
  function setStatus(s: string) {
    const q = new URLSearchParams(search);
    q.set("status", s);
    router.push(`${pathname}?${q}`); // also: router.replace, router.back, router.refresh
  }
  return <button onClick={() => setStatus("OPEN")}>Open</button>;
}
```

TSX

```
// Server-side redirects
import { redirect, permanentRedirect, notFound } from "next/navigation";
if (!session) redirect("/login");
```

Tool	Runs in	Use
<code><Link></code>	Both	Standard navigation
<code>useRouter()</code>	Client Component	Programmatic navigation
<code>redirect()</code>	Server (components, actions, handlers)	Redirect during render or after a mutation
<code>usePathname</code> , <code>useSearchParams</code>	Client Component	Read the current URL (wrap <code>useSearchParams</code> in <code><Suspense></code>)

2.4.1 Loading and error UI

TSX

```
// app/tasks/loading.tsx - shown immediately while /tasks streams its data
export default function Loading() {
  return <div className="animate-pulse space-y-3">
    <div className="h-6 w-1/3 rounded bg-gray-200" />
    <div className="h-4 w-full rounded bg-gray-200" />
  </div>;
}

// app/tasks/error.tsx - must be a Client Component
"use client";
export default function Error({ error, reset }: { error: Error & { digest?: string }; reset: () => void }) {
  return (
    <div role="alert" className="rounded border border-red-300 bg-red-50 p-4">
      <p className="font-medium text-red-800">Something went wrong.</p>
      <button onClick={reset} className="mt-2 rounded bg-red-600 px-3 py-1 text-white">Try again</button>
    </div>
  );
}

// app/not-found.tsx
export default function NotFound() { return <h1>404 - Page not found</h1>; }
```

2.5 Metadata and SEO

TSX

```
// Static
export const metadata: Metadata = {
  title: "Tasks",
  description: "Your tasks",
  openGraph: { title: "Tasks", images: ["/og.png"] },
  robots: { index: true, follow: true },
};

// Dynamic (per page data)
export async function generateMetadata({ params }: { params: Promise<{ id: string }> }): Promise<Metadata> {
  const { id } = await params;
  const task = await getTask(id); // fetch is deduplicated with the page's own call
  return { title: task.title, alternates: { canonical: `/tasks/${id}` } };
}
```

File-based metadata: put `favicon.ico`, `icon.png`, `opengraph-image.tsx`, `robots.ts`, `sitemap.ts` in `app/`.

2.6 Images, fonts, scripts

TSX

```
import Image from "next/image";

<Image
  src="/hero.jpg" alt="Team working"
  width={1200} height={630} // required for local/remote images (prevents layout shift)
  priority // preload above-the-fold (LCP) image
  sizes="(max-width: 768px) 100vw, 50vw"
  className="rounded-xl"
/>

// Remote images must be allowed in next.config.ts:
// images: { remotePatterns: [{ protocol: "https", hostname: "cdn.example.com" }] }
```

`next/image` automatically serves modern formats (AVIF/WebP), correct sizes per device, lazy loading and reserved space. `next/font` self-hosts fonts at build time (no request to Google at runtime). `next/script` loads third-party scripts with strategies (`afterInteractive`, `lazyOnload`).

2.7 Environment variables

File	Loaded
<code>.env</code>	All environments
<code>.env.local</code>	Local overrides (never commit)
<code>.env.development</code> / <code>.env.production</code>	Per mode

BASH

```
# .env.local
API_URL=http://localhost:8080      # server only
DATABASE_URL=postgres://...      # server only
NEXT_PUBLIC_APP_NAME=TaskFlow     # exposed to the browser (inlined at build time)
```

WARNING

Only variables prefixed with `NEXT_PUBLIC_` reach the browser. Never prefix secrets with it. Use the `server-only` package (`import "server-only"`) in modules with secrets so importing them from a Client Component fails the build.

2.8 Configuration

TS

```
// next.config.ts
import type { NextConfig } from "next";

const config: NextConfig = {
  reactStrictMode: true,
  output: "standalone", // minimal server bundle for Docker
  reactCompiler: true, // enable the React Compiler (stable in v16)
  cacheComponents: true, // enable Cache Components / "use cache" (v16)
  images: { remotePatterns: [{ protocol: "https", hostname: "cdn.example.com" }] },
  async rewrites() {
    return [{ source: "/backend/:path*", destination: "http://localhost:8080/api/:path*" }]; // proxy to Spring
  },
  async headers() {
    return [{ source: "/(.*)", headers: [{ key: "X-Frame-Options", value: "DENY" }] }];
  },
};
export default config;
```

3. Server and Client Components

CATEGORY

Core concept. The single most important idea in modern Next.js: which code runs on the server, which in the browser, and how they compose. Level: Beginner to Intermediate.

3.1 The two kinds of components

	Server Component (default)	Client Component ("use client")
Runs	On the server only (build or request time)	Server pre-render + browser (hydrated)
Can be <code>async</code> and <code>await</code> data	Yes	No
Access DB, secrets, file system	Yes	No
<code>useState</code> , <code>useEffect</code> , <code>useRef</code> , context	No	Yes
Event handlers (<code>onClick</code>)	No	Yes
Browser APIs (<code>window</code> , <code>localStorage</code>)	No	Yes
JavaScript sent to browser	None	Yes (adds to bundle)
Best for	Layouts, data display, static content	Forms, buttons, menus, charts, interactivity

TEXT

```
Server Component tree (no JS shipped)
├── Page (server, async, queries DB)
│   ├── Header (server)
│   ├── TaskTable (server) → renders rows as HTML
│   └── AddTaskButton + "use client" boundary: this and everything IMPORTED by it is client code
│       └── Modal (client)
```

3.2 The "use client" boundary

TSX

```
// components/LikeButton.tsx
"use client"; // must be at the very top of the file

import { useState } from "react";

export function LikeButton({ initial }: { initial: number }) {
  const [likes, setLikes] = useState(initial);
  return (
    <button onClick={() => setLikes(likes + 1)} className="rounded bg-pink-500 px-3 py-1 text-white">
      Like {likes}
    </button>
  );
}
```

TSX

```
// app/posts/[id]/page.tsx (Server Component)
import { LikeButton } from "@components/LikeButton";

export default async function Post({ params }: { params: Promise<{ id: string }> }) {
  const { id } = await params;
  const post = await db.post.findUnique({ where: { id: Number(id) } }); // direct DB access
  return (
    <article>
      <h1>{post.title}</h1>
      <p>{post.body}</p>
      <LikeButton initial={post.likes} />      {/* server data passed as props to a client component */}
    </article>
  );
}
```

3.3 Composition rules

1. A Server Component **can import and render** Client Components.
2. A Client Component **cannot import** a Server Component, but it can **receive one as children or a prop** (composition pattern).
3. Props passed from server to client must be **serialisable** (strings, numbers, plain objects, arrays, Dates, Promises, Server Actions). No functions (except Server Actions), class instances or symbols.
4. Put **"use client"** as deep in the tree as possible so most of the page stays server-rendered.

TSX

```
// Composition: interactive shell + server-rendered content inside
// components/Collapsible.tsx
"use client";
export function Collapsible({ title, children }: { title: string; children: React.ReactNode }) {
  const [open, setOpen] = useState(false);
  return <div><button onClick={() => setOpen(!open)}>{title}</button>{open && children}</div>;
}
// page.tsx (server)
<Collapsible title="Comments"><Comments postId={id} /></Collapsible>      {/* Comments stays a Server Component */}
```

3.4 Context providers

Context requires a Client Component. Wrap it in a small provider and use it in the (server) root layout.

TSX

```
// app/providers.tsx
"use client";
import { ThemeProvider } from "next-themes";
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import { useState } from "react";

export function Providers({ children }: { children: React.ReactNode }) {
  const [client] = useState(() => new QueryClient());
  return (
    <ThemeProvider attribute="class" defaultTheme="system" enableSystem>
      <QueryClientProvider client={client}>{children}</QueryClientProvider>
    </ThemeProvider>
  );
}

// app/layout.tsx (server): <body><Providers>{children}</Providers></body>
```

3.5 Server-only and client-only code

TS

```
// lib/db.ts
import "server-only"; // build error if a client component imports this module
import { PrismaClient } from "@prisma/client";
export const db = globalThis.prisma ?? new PrismaClient(); // reuse the client during dev hot reload
```

3.6 When to choose which - quick guide

You need...	Use
Fetch data and render it	Server Component
Read cookies/headers, check auth	Server Component or Server Action
Click, input, toggle, drag, animation state	Client Component
<code>useEffect</code> , <code>localStorage</code> , <code>window</code>	Client Component
A third-party UI library that uses hooks	Client Component wrapper
Keep an API key secret	Server only

4. Next.js - Data Fetching, Rendering and Caching

CATEGORY

Framework: data and performance. Fetching in Server Components, parallel vs sequential requests, streaming with Suspense, static vs dynamic rendering, and the Next.js 16 caching model. Level: Intermediate.

4.1 Fetching data in a Server Component

TSX

```
// app/tasks/page.tsx
type Task = { id: number; title: string; status: "OPEN" | "DONE" };

async function getTasks(status?: string): Promise<Task[]> {
  const url = new URL("/api/tasks", process.env.API_URL); // your Spring Boot / Node API
  if (status) url.searchParams.set("status", status);
  const res = await fetch(url, { headers: { Authorization: `Bearer ${await getToken()}` } });
  if (!res.ok) throw new Error(`API error ${res.status}`); // caught by error.tsx
  const page = await res.json();
  return page.content;
}

export default async function TasksPage({ searchParams }: { searchParams: Promise<{ status?: string }> }) {
  const { status } = await searchParams;
  const tasks = await getTasks(status);
  return (
    <ul className="divide-y rounded-lg border">
      {tasks.map((t) => (
        <li key={t.id} className="flex items-center justify-between p-3">
          <span className={t.status === "DONE" ? "text-gray-400 line-through" : ""}>{t.title}</span>
          <span className="text-xs uppercase text-gray-500">{t.status}</span>
        </li>
      ))}
    </ul>
  );
}
```

You can also query a database or ORM directly (`await db.task.findMany()`), because Server Components run only on the server. Compare with Vol 2, where the same data required `useQuery` in the browser.

4.2 Sequential vs parallel fetching

TSX

```
// SLOW: waterfall - the second request starts only after the first finishes
const user = await getUser(id);
const orders = await getOrders(user.id); // (needed only if orders truly depend on user)

// FAST: independent requests run in parallel
const [user, stats, notifications] = await Promise.all([getUser(id), getStats(id), getNotifications(id)]);

// FASTEST perceived: stream independent sections with Suspense (below)
```

4.3 Streaming with Suspense

Wrap slow parts in `<Suspense>`: the fast parts of the page appear immediately and the slow parts stream in when ready.

TSX

```
import { Suspense } from "react";

export default function Dashboard() {
  return (
    <div className="grid gap-6 md:grid-cols-2">
      <h1 className="col-span-full text-2xl font-bold">Dashboard</h1>    {/* instantly visible */}

      <Suspense fallback={<CardSkeleton />}>
        <RevenueChart />          {/* async Server Component: slow query */}
      </Suspense>

      <Suspense fallback={<CardSkeleton />}>
        <RecentOrders />          {/* independent: streams separately */}
      </Suspense>
    </div>
  );
}

async function RevenueChart() {
  const data = await getRevenue();    // takes 2 s - does not block the rest of the page
  return <Chart data={data} />;
}
```

`loading.tsx` is the same idea applied to a whole route segment.

4.4 Static vs dynamic rendering

Mode	When HTML is produced	Triggered by	Best for
Static	At build time (cached)	No request-time data used	Marketing pages, docs, blog
Dynamic	On every request	<code>cookies()</code> , <code>headers()</code> , <code>searchParams</code> , uncached <code>fetch</code> , <code>connection()</code>	Personalised pages, dashboards
Partial (Cache Components)	Static shell + dynamic holes streamed	"use cache" + Suspense boundaries	Most real pages: fast shell, fresh parts
ISR (time-based revalidate)	Static, regenerated in the background	<code>cacheLife</code> / <code>revalidate</code>	Catalogues, news

TSX

```
// Pre-render dynamic routes at build time
export async function generateStaticParams() {
  const posts = await getAllPosts();
  return posts.map((p) => ({ id: String(p.id) }));    // /posts/1, /posts/2, ...
}
export const dynamicParams = true;                  // other ids rendered on demand, then cached
```

4.5 Caching model in Next.js 16 (Cache Components)

Previous versions cached `fetch` results implicitly, which surprised many developers. Next.js 15 made `fetch`, `GET` handlers and client router cache **uncached by default**. Next.js 16 makes caching **explicit and opt-in**: all dynamic code runs at request time unless you cache it with the `"use cache"` directive (enable with `cacheComponents: true` in `next.config.ts`).

TSX

```
import { cacheLife, cacheTag } from "next/cache";

// Cache a function's result (arguments become part of the cache key)
export async function getProducts() {
  "use cache";
  cacheLife("hours"); // built-in profiles: seconds, minutes, hours, days, weeks, max
  cacheTag("products"); // label to invalidate on demand
  const res = await fetch(`${process.env.API_URL}/api/products`);
  return res.json();
}

// Cache a whole component
export async function Header() {
  "use cache";
  cacheLife("days");
  const nav = await getNavigation();
  return <nav>{/* ... */</nav>;
}

// Custom lifetime
cacheLife({ stale: 60, revalidate: 300, expire: 3600 }); // seconds
```

Rules for **"use cache"**: it cannot read `cookies()` / `headers()` / `searchParams` directly (read them outside and pass values as arguments); arguments and return values must be serialisable; wrap dynamic (uncached) parts in `<Suspense>`.

Invalidation

TS

```
"use server";
import { revalidateTag, updateTag, revalidatePath, refresh } from "next/cache";

export async function createProduct(data: FormData) {
  await api.create(data);
  updateTag("products"); // Server Actions only: expire AND read-your-writes (user sees new
    data immediately)
  // revalidateTag("products", "max"); // stale-while-revalidate style: serve stale, refresh in background
  // revalidatePath("/products"); // invalidate by URL
  // refresh(); // refresh uncached data of the current route
}
```

API	Effect
"use cache"	Cache a component / function / page
<code>cacheLife(profile)</code>	How long cached data stays fresh
<code>cacheTag(tag)</code>	Attach tags to cache entries
<code>updateTag(tag)</code>	Immediately expire tag inside a Server Action (user sees own changes)
<code>revalidateTag(tag, profile)</code>	Mark tag stale; refresh in the background (SWR)
<code>revalidatePath(path)</code>	Invalidate a route
<code>refresh()</code>	Re-render current route's uncached data

4.6 Request-time APIs

TSX

```
import { cookies, headers, draftMode } from "next/headers";

export default async function Profile() {
  const cookieStore = await cookies(); // async in v15+
  const theme = cookieStore.get("theme")?.value ?? "light";
  const h = await headers();
  const ua = h.get("user-agent");
  // Using these makes the route dynamic (request-time) unless isolated behind Suspense
}
```

4.7 Client-side fetching is still useful for...

- Data that changes constantly on the client (live search, polling, infinite scroll) - use **TanStack Query** or **SWR** in a Client Component.
- Data behind user interaction after the page loaded.
- Pass initial data from the server and let the client take over (hydration of the query cache).

5. Next.js - Mutations, Server Actions and Route Handlers

CATEGORY

Framework: writing data and building endpoints. Server Actions for forms and mutations, Route Handlers for HTTP APIs, and `proxy.ts` for request interception. Level: Intermediate.

5.1 Server Actions

A **Server Action** is an async function marked `"use server"` that runs on the server but can be called directly from a form or a Client Component. Next.js creates a hidden POST endpoint for it automatically.

TS

```
// app/tasks/actions.ts
"use server";

import { z } from "zod";
import { revalidatePath } from "next/cache";
import { redirect } from "next/navigation";

const CreateTask = z.object({
  title: z.string().trim().min(1, "Title is required").max(100),
  priority: z.coerce.number().int().min(1).max(5),
});

export type FormState = { errors?: Record<string, string[]>; message?: string } | undefined;

export async function createTask(_prev: FormState, formData: FormData): Promise<FormState> {
  const session = await getSession(); // ALWAYS authenticate inside the
  action
  if (!session) return { message: "Not authorised" };

  const parsed = CreateTask.safeParse(Object.fromEntries(formData)); // ALWAYS validate on the server
  if (!parsed.success) return { errors: parsed.error.flatten().fieldErrors };

  const res = await fetch(`${process.env.API_URL}/api/tasks`, {
    method: "POST",
    headers: { "Content-Type": "application/json", Authorization: `Bearer ${session.token}` },
    body: JSON.stringify(parsed.data),
  });
  if (!res.ok) return { message: "Could not create task" };

  revalidatePath("/tasks"); // refresh the list
  redirect("/tasks"); // navigate (redirect throws internally: call it outside try/catch)
}
```

TSX

```
// app/tasks/new/task-form.tsx
"use client";
import { useActionState } from "react";
import { createTask } from "../actions";

export function TaskForm() {
  const [state, action, pending] = useActionState(createTask, undefined);
  return (
    <form action={action} className="space-y-3">
      <div>
        <input name="title" placeholder="Title" className="w-full rounded border p-2" />
        {state?.errors?.title && <p className="text-sm text-red-600">{state.errors.title[0]}</p>}
      </div>
      <select name="priority" defaultValue="3" className="rounded border p-2">
        {[1, 2, 3, 4, 5].map((n) => <option key={n}>{n}</option>)}
      </select>
      <button disabled={pending} className="rounded bg-blue-600 px-4 py-2 text-white disabled:opacity-50">
        {pending ? "Saving..." : "Create"}
      </button>
      {state?.message && <p role="alert" className="text-red-600">{state.message}</p>}
    </form>
  );
}
```

TSX

```
// Inline Server Action in a Server Component (progressive enhancement: works without JS)
export default async function Page() {
  async function remove(formData: FormData) {
    "use server";
    await db.task.delete({ where: { id: Number(formData.get("id")) } });
    revalidatePath("/tasks");
  }
  return <form action={remove}><input type="hidden" name="id" value="5" /><button>Delete</button></form>;
}
```

WARNING

Server Actions are **public HTTP endpoints**. Treat every argument as untrusted user input: authenticate, authorise (does this user own this task?) and validate with Zod inside every action. Do not rely on the UI hiding a button.

5.2 Server Actions vs Route Handlers

	Server Action	Route Handler (<i>route.ts</i>)
Purpose	Mutations from your own UI	Public / external HTTP API
Called from	Forms, Client Components (<i>action</i> , <i>startTransition</i>)	Any HTTP client (mobile app, webhooks, third parties)
HTTP verb	POST only (internal)	GET, POST, PUT, PATCH, DELETE...
Response	Serialised return value, redirect, revalidation	Any <i>Response</i> (JSON, stream, file)
Progressive enhancement	Yes (forms work without JS)	N/A
Typical use	Create/update/delete then refresh UI	Webhooks (Stripe), file download, REST API for other clients

5.3 Route Handlers

TS

```
// app/api/tasks/route.ts -> /api/tasks
import { NextRequest, NextResponse } from "next/server";
import { z } from "zod";

export async function GET(req: NextRequest) {
  const status = req.nextUrl.searchParams.get("status"); // ?status=OPEN
  const tasks = await db.task.findMany({ where: status ? { status } : {} });
  return NextResponse.json(tasks); // GET is uncached by default (v15+)
}

const Body = z.object({ title: z.string().min(1), priority: z.number().int().min(1).max(5) });

export async function POST(req: NextRequest) {
  const parsed = Body.safeParse(await req.json());
  if (!parsed.success) return NextResponse.json({ errors: parsed.error.flatten() }, { status: 400 });
  const task = await db.task.create({ data: parsed.data });
  return NextResponse.json(task, { status: 201, headers: { Location: `/api/tasks/${task.id}` } });
}

// app/api/tasks/[id]/route.ts -> /api/tasks/42 (params is a Promise)
export async function DELETE(_req: NextRequest, { params }: { params: Promise<{ id: string }> }) {
  const { id } = await params;
  await db.task.delete({ where: { id: Number(id) } });
  return new NextResponse(null, { status: 204 });
}
```

TS

```
// Webhook example with raw body verification
export async function POST(req: Request) {
  const raw = await req.text();
  const sig = req.headers.get("stripe-signature");
  const event = stripe.webhooks.constructEvent(raw, sig, process.env.STRIPE_WEBHOOK_SECRET!);
  // handle event...
  return new Response("ok");
}
```

5.4 proxy.ts (formerly middleware.ts)

proxy.ts runs **before** a request is completed: redirect, rewrite, set headers, do optimistic auth checks. In Next.js 16 the file is called **proxy.ts** and exports a function named **proxy** (it runs on the Node.js runtime).

TS

```
// proxy.ts (project root, next to app/)
import { NextRequest, NextResponse } from "next/server";
import { jwtVerify } from "jose";

const secret = new TextEncoder().encode(process.env.JWT_SECRET);
const PUBLIC = ["/", "/login", "/register"];

export async function proxy(req: NextRequest) {
  const { pathname } = req.nextUrl;
  if (PUBLIC.includes(pathname)) return NextResponse.next();

  const token = req.cookies.get("session")?.value;
  try {
    if (!token) throw new Error("no token");
    await jwtVerify(token, secret);
    return NextResponse.next();
  } catch {
    const url = new URL("/login", req.url);
    url.searchParams.set("from", pathname);
    return NextResponse.redirect(url);
  }
}

export const config = { matcher: ["/((?!api|_next/static|_next/image|favicon.ico).*)"] };
}
```

NOTE

Use **proxy.ts** for **coarse, optimistic checks** (is there a session cookie?). Perform the real authorisation in the data access layer close to the data (Server Components, Server Actions, Route Handlers) so a missed matcher cannot expose data.

5.5 Comparison: same endpoint in three stacks

JAVA

```
// Spring Boot
@PostMapping("/api/tasks")
ResponseEntity<Task> create(@Valid @RequestBody CreateTask dto) {
    Task t = service.create(dto);
    return ResponseEntity.created(URI.create("/api/tasks/" + t.id())).body(t);
}
```

TS

```
// Express
app.post("/api/tasks", async (req, res) => {
    const dto = CreateTask.parse(req.body);
    const t = await service.create(dto);
    res.status(201).location(`/api/tasks/${t.id}`).json(t);
});
```

TS

```
// Next.js Route Handler
export async function POST(req: Request) {
    const dto = CreateTask.parse(await req.json());
    const t = await service.create(dto);
    return Response.json(t, { status: 201, headers: { Location: `/api/tasks/${t.id}` } });
}
```

6. Next.js - Advanced

CATEGORY

Framework: production architecture. Authentication, data access layer, parallel and intercepting routes, runtimes, performance, security, testing, deployment and using Next.js as a backend-for-frontend for Spring Boot. Level: Advanced.

6.1 Authentication and authorisation

Option	Description	When
Auth.js (NextAuth v5)	OAuth/OIDC providers, credentials, sessions, adapters	Social login quickly
Better Auth / Lucia-style custom	Library or hand-rolled sessions	Full control
Clerk / Auth0 / Supabase Auth	Hosted identity	Fastest to production
Your Spring Boot API issues JWT	Next.js stores the token in an httpOnly cookie	Existing Java backend / enterprise

TS

```
// lib/session.ts - stateless session in an httpOnly cookie
import "server-only";
import { SignJWT, jwtVerify } from "jose";
import { cookies } from "next/headers";
import { cache } from "react";

const key = new TextEncoder().encode(process.env.SESSION_SECRET);
export async function createSession(userId: string, roles: string[]) {
  const token = await new SignJWT({ userId, roles }).setProtectedHeader({ alg: "HS256" })
    .setIssuedAt().setExpirationTime("7d").sign(key);
  (await cookies()).set("session", token, {
    httpOnly: true, secure: true, sameSite: "lax", path: "/", maxAge: 60 * 60 * 24 * 7,
  });
}
export const getSession = cache(async () => { // React cache(): one verification per request
  const token = (await cookies()).get("session")?.value;
  if (!token) return null;
  try { return (await jwtVerify(token, key)).payload as { userId: string; roles: string[] }; }
  catch { return null; }
});
```

TS

```
// lib/dal.ts - Data Access Layer: every data function authorises itself
import "server-only";
export async function getMyTasks() {
  const s = await getSession();
  if (!s) redirect("/login");
  return db.task.findMany({ where: { ownerId: s.userId } }); // authorisation lives next to the data
}
```

6.2 Parallel and intercepting routes (modals with URLs)

TEXT

```

app/
  layout.tsx      <- receives {children, modal} slots
  @modal/
    default.tsx   <- returns null when no modal is open
    (...)photos/[id]/page.tsx <- intercepts /photos/[id] and shows it in a modal over the current page
  photos/
    page.tsx      <- gallery
    [id]/page.tsx <- full page (opens on refresh or direct link)

```

Clicking a photo in the gallery shows a **modal**; refreshing or sharing the URL shows the **full page**. Both are the same URL. Parallel routes (**@slot**) also power dashboards with independently loading panels and independent error states.

6.3 Runtimes

Runtime	Characteristics	Use
Node.js (default)	Full Node APIs, all npm packages	Almost everything
Edge	V8 isolates, tiny, global, no Node APIs, size limits	Ultra-low latency logic (simple redirects, geolocation)

TS

```

export const runtime = "nodejs";      // or "edge" in a route segment config
export const maxDuration = 30;        // seconds (platform dependent)

```

6.4 Performance checklist

1. Prefer **Server Components**; keep "use client" boundaries small and deep.
2. Use **next/image** with correct **sizes**, **priority** only for the LCP image.
3. Use **next/font** to avoid layout shift and font requests.
4. **Stream** slow data with **<Suspense>** and **loading.tsx**; fetch in parallel.
5. Cache with "use cache" and **cacheLife**; tag data to invalidate precisely.
6. **Dynamic import** heavy client-only code: `const Chart = dynamic(() => import("../Chart"), { ssr: false, loading: () => <Skeleton /> })`.
7. Analyse bundles with **@next/bundle-analyzer**; avoid importing whole icon or utility libraries (**import { debounce } from "lodash-es"**).
8. Enable the **React Compiler** (`reactCompiler: true`).
9. Measure **Core Web Vitals** (LCP, INP, CLS) with **useReportWebVitals**, Lighthouse and real-user monitoring.
10. Put images and static files behind a CDN; use **Cache-Control** headers for immutable assets (automatic for **_next/static**).

6.5 Security checklist

Area	Practice
Secrets	Only NEXT_PUBLIC_ variables reach the browser; use server-only
Server Actions	Authenticate, authorise and validate every call; treat as public endpoints
Auth	httpOnly , Secure , SameSite cookies; verify sessions in the data layer, not only in proxy.ts
Headers	Set CSP , X-Content-Type-Options , Referrer-Policy , Strict-Transport-Security in next.config.ts or proxy.ts
XSS	React escapes output; never use dangerouslySetInnerHTML with untrusted HTML (sanitise first)

Area	Practice
CSRF	Server Actions compare Origin and Host ; cookies with SameSite; configure allowedOrigins if behind proxies
SSRF	Validate URLs before server-side fetches; restrict remotePatterns
Dependencies	npm audit , keep Next.js patched, lock versions
Rate limiting	Apply at the edge/CDN or in the API (login, actions)

6.6 Testing strategy

Layer	Tool	Notes
Utilities, hooks, client components	Vitest + Testing Library	Standard
Async Server Components	E2E tests (Playwright)	Unit-testing them is limited in tooling
Route Handlers / Server Actions	Vitest (call the function directly)	Mock the DB/API
End to end	Playwright	Real browser, real routes, auth flows
Visual	Storybook / Chromatic / Playwright screenshots	UI regression

TS

```
// e2e/tasks.spec.ts (Playwright)
import { test, expect } from "@playwright/test";

test("create a task", async ({ page }) => {
  await page.goto("/tasks/new");
  await page.getByPlaceholder("Title").fill("Write docs");
  await page.getByRole("button", { name: "Create" }).click();
  await expect(page).toHaveURL("/tasks");
  await expect(page.getByText("Write docs")).toBeVisible();
});
```

6.7 Deployment options

Option	How	Pros	Cons
Vercel	Git push	Zero config, best Next.js feature support, edge network	Cost at scale, platform lock-in
Docker / VPS / Kubernetes	output: "standalone" + Node image	Full control, any cloud	You manage scaling, caching, ISR storage
AWS (Amplify, SST/OpenNext), Cloudflare, Netlify	Adapters	Fits existing cloud	Some features lag behind Vercel
Static export	output: "export"	Host anywhere (S3, GitHub Pages)	No server features (actions, dynamic rendering)

DOCKERFILE

```
FROM node:22-slim AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build          # with output: "standalone"

FROM node:22-slim
ENV NODE_ENV=production PORT=3000
WORKDIR /app
COPY --from=build /app/.next/standalone ./
COPY --from=build /app/.next/static ./next/static
COPY --from=build /app/public ./public
USER node
EXPOSE 3000
CMD ["node", "server.js"]
```

6.8 Next.js as a Backend-for-Frontend (BFF) for Spring Boot

TEXT

```
Browser <--HTML/RSC--> Next.js (server) <--REST/JSON + service token--> Spring Boot services
| - auth cookie <=> JWT translation
| - aggregates 3 API calls into one page
| - hides internal API URLs and secrets
| - caches with "use cache" and tags
```

Benefits: the browser never sees the internal API or tokens, fewer round trips, one language for UI and glue code, Spring keeps the core domain, transactions and heavy processing. Use a **typed API client** generated from the Spring OpenAPI document ([openapi-typescript](#) or [orval](#)) so frontend types always match the backend contract.

BASH

```
npx openapi-typescript http://localhost:8080/v3/api-docs -o src/lib/api-types.d.ts
```

6.9 Other Next.js features worth knowing

- **Internationalisation:** locale route segment `app/[lang]/...` with `next-intl` or `il18next`.
- **Draft mode / preview** for headless CMS content.
- **Instrumentation:** `instrumentation.ts` with OpenTelemetry for traces and metrics.
- **Typed routes:** compile-time checking of `href` strings.
- **Turbopack:** default bundler; file-system caching (beta) speeds restarts. Use `--webpack` only if a plugin needs it.
- **After:** `after(() => log(...))` runs work after the response is sent (analytics, logging).
- `next/og`: generate Open Graph images with JSX.
- **Migrating:** from Pages Router incrementally (both can coexist); codemods via `npx @next/codemod@latest`.

7. Tailwind CSS - Basics

CATEGORY

Styling: utility-first CSS framework. Style elements by combining small single-purpose classes directly in markup. Used with React, Next.js, Vue, plain HTML. Level: Beginner.

7.1 The idea

Traditional CSS: invent class names (`.card`, `.card-title`), write rules in a stylesheet, switch between files. **Tailwind**: apply ready-made utilities such as `p-4`, `text-lg`, `bg-blue-600` in the element itself. A build step scans your files and generates **only the CSS you actually used**, so the final stylesheet is tiny.

HTML

```
<!-- Traditional CSS -->
<style>
  .btn { padding: 0.5rem 1rem; background: #2563eb; color: white; border-radius: 0.375rem; }
  .btn:hover { background: #1d4ed8; }
</style>
<button class="btn">Save</button>

<!-- Tailwind -->
<button class="rounded-md bg-blue-600 px-4 py-2 text-white hover:bg-blue-700">Save</button>
```

Advantage	Detail
No naming fatigue	No <code>.wrapper-inner-left-2</code> class names
Local reasoning	Styles live where the element is; deleting a component deletes its styles
Consistent design	Spacing, colours and sizes come from a shared scale (design tokens)
Small CSS output	Unused utilities are never generated
Responsive and state variants built in	<code>md:</code> , <code>hover:</code> , <code>dark:</code> prefixes
Great with components	React/Vue components remove repetition

Common criticism: long class lists. Answer: extract **components**, not CSS classes; use editor plugins (Prettier plugin sorts classes) and `cva/tailwind-variants` for variants.

7.2 Install (Tailwind v4)

BASH

```
# With Next.js: selected automatically by create-next-app. Manual install:
npm install tailwindcss @tailwindcss/postcss postcss      # Next.js / PostCSS projects
npm install tailwindcss @tailwindcss/vite                  # Vite projects
```

JS

```
// postcss.config.mjs (Next.js)
export default { plugins: { "@tailwindcss/postcss": {} } };
```

TS

```
// vite.config.ts (Vite)
import tailwindcss from "@tailwindcss/vite";
export default defineConfig({ plugins: [react(), tailwindcss()] });
```

CSS

```
/* app/globals.css (v4: one line replaces @tailwind base/components/utilities) */
@import "tailwindcss";
```

v4 detects your source files automatically (no `content` array) and needs no `tailwind.config.js` for most projects. Customisation is done in CSS with `@theme` (Chapter 9).

7.3 Core utilities by category

Spacing (padding, margin, gap)

The default scale is a multiple of 0.25rem: `1 = 4px`, `2 = 8px`, `4 = 16px`, `8 = 32px`.

Class	CSS
<code>p-4</code>	<code>padding: 1rem</code>
<code>px-6 py-2</code>	horizontal 1.5rem, vertical 0.5rem
<code>pt-2 pr-3 pb-4 pl-5</code>	single sides
<code>m-4, mx-auto, -mt-2</code>	margin, centre horizontally, negative margin
<code>space-y-4</code>	vertical gap between children
<code>gap-4, gap-x-2</code>	gap in flex/grid
<code>p-[13px]</code>	arbitrary value

Typography

Class	Effect
<code>text-sm text-base text-lg text-2xl</code>	Font size (with sensible line height)
<code>font-light font-medium font-semibold font-bold</code>	Weight
<code>text-gray-600 text-blue-500</code>	Colour (scale 50-950)
<code>text-center text-left</code>	Alignment
<code>leading-tight leading-relaxed</code>	Line height
<code>tracking-wide</code>	Letter spacing
<code>uppercase capitalize truncate line-clamp-2</code>	Transform, single-line ellipsis, multi-line clamp
<code>font-sans font-serif font-mono</code>	Font family

Colours, backgrounds, borders, effects

Class	Effect
<code>bg-white bg-gray-100 bg-blue-600/50</code>	Background; <code>/50</code> is 50% opacity
<code>bg-gradient-to-r from-pink-500 to-yellow-500</code>	Gradient (v4 also: <code>bg-linear-to-r</code>)
<code>border border-2 border-gray-200</code>	Border width and colour
<code>rounded rounded-lg rounded-full</code>	Border radius
<code>shadow shadow-md shadow-xl</code>	Box shadow
<code>ring-2 ring-blue-500 ring-offset-2</code>	Outline-like ring (focus styles)
<code>opacity-50</code>	Opacity
<code>divide-y divide-gray-200</code>	Borders between children

Sizing

Class	Effect
w-full w-1/2 w-64 w-[300px]	Width
h-screen h-10 min-h-screen	Height
max-w-md max-w-3xl max-w-7xl	Max width (content containers)
size-10	Width and height together
aspect-video aspect-square	Aspect ratio

7.4 Layout: Flexbox

HTML

```
<div class="flex items-center justify-between gap-4">      <!-- row, vertically centred, space between -->
  <h1 class="text-xl font-bold">Logo</h1>
  <nav class="flex gap-6 text-sm"><a>Home</a><a>Docs</a><a>Pricing</a></nav>
</div>

<div class="flex flex-col gap-2">...</div>                <!-- column -->
<div class="flex flex-wrap gap-2">...</div>               <!-- wrap items -->
<div class="flex min-h-screen items-center justify-center">...</div>  <!-- centre anything -->
<div class="flex-1">...</div>                                     <!-- grow to fill space -->
```

Class	CSS
flex / inline-flex	display: flex
flex-row / flex-col	direction
items-start center end stretch	align-items
justify-start center end between around evenly	justify-content
flex-1 grow shrink-0	flex sizing
order-first order-last	order

7.5 Layout: Grid

HTML

```
<div class="grid grid-cols-3 gap-4">                        <!-- 3 equal columns -->
  <div>1</div><div>2</div><div class="col-span-2">wide</div>
</div>

<!-- Responsive card grid: 1 column on phones, 2 on tablets, 4 on desktops -->
<div class="grid grid-cols-1 gap-6 sm:grid-cols-2 lg:grid-cols-4">
  <article class="rounded-xl border p-4">Card</article>
</div>

<!-- Auto-fit grid without breakpoints -->
<div class="grid grid-cols-[repeat(auto-fit,minmax(16rem,1fr))] gap-4">...</div>
```

Flexbox vs Grid	Use
Flexbox	One-dimensional: navbars, button rows, centring, toolbars
Grid	Two-dimensional: page layouts, card grids, dashboards

7.6 Display, position and overflow

Class	Effect
block inline-block hidden	Display
relative absolute fixed sticky	Position
top-0 right-4 inset-0 inset-x-0	Offsets

Class	Effect
<code>z-10 z-50</code>	Stacking
<code>overflow-hidden overflow-auto overflow-x-scroll</code>	Overflow
<code>container mx-auto px-4</code>	Centred container with breakpoint max widths
<code>sticky top-0 z-40 bg-white/80 backdrop-blur</code>	Sticky translucent header

7.7 Responsive design (mobile first)

Unprefixed utilities apply to **all** screen sizes. A prefix means "from this breakpoint **and up**".

Prefix	Min width	Typical device
(none)	0	Phones (default)
<code>sm:</code>	640px	Large phones
<code>md:</code>	768px	Tablets
<code>lg:</code>	1024px	Laptops
<code>xl:</code>	1280px	Desktops
<code>2xl:</code>	1536px	Large screens
<code>max-md:</code>	below 768px	Target only smaller screens

HTML

```
<div class="flex flex-col gap-4 md:flex-row md:gap-8">      <!-- column on mobile, row from tablet -->
  <aside class="w-full md:w-64">Sidebar</aside>
  <section class="flex-1">Content</section>
</div>
<h1 class="text-2xl sm:text-3xl lg:text-5xl">Fluid heading</h1>
<nav class="hidden md:flex">Desktop menu</nav>
<button class="md:hidden">Hamburger</button>
```

7.8 State variants

HTML

```
<button class="bg-blue-600 text-white
  hover:bg-blue-700
  focus-visible:outline focus-visible:outline-2 focus-visible:outline-offset-2
  focus-visible:outline-blue-600
  active:scale-95
  disabled:cursor-not-allowed disabled:opacity-50
  transition-colors duration-150">
  Save
</button>

<input class="border border-gray-300 focus:border-blue-500 focus:ring-2 focus:ring-blue-200
  invalid:border-red-500 placeholder:text-gray-400" />

<li class="odd:bg-gray-50 first:rounded-t last:rounded-b">Row</li>
```

Parent/sibling state: mark the parent with `group` and style children with `group-hover:;`; mark an element `peer` and style the next sibling with `peer-checked:` or `peer-invalid:.`

HTML

```
<a class="group flex items-center gap-2 rounded p-2 hover:bg-gray-100">
  <span class="text-gray-500 group-hover:text-blue-600">Link</span>
  <svg class="translate-x-0 transition group-hover:translate-x-1">...</svg>
</a>

<input type="checkbox" id="agree" class="peer sr-only" />
<label for="agree" class="rounded border p-2 peer-checked:border-blue-600 peer-checked:bg-blue-50">I
  agree</label>
```

Other variants: `aria-expanded:`, `data-[state=open]:`, `has-[:checked]:`, `not-last:`, `motion-reduce:`, `print:`, `rtl:`, `*`: (all direct children).

7.9 Dark mode

HTML

```
<div class="bg-white text-gray-900 dark:bg-gray-900 dark:text-gray-100">
  <h1 class="text-gray-800 dark:text-white">Title</h1>
</div>
```

By default `dark:` follows the OS setting (`prefers-color-scheme`). To toggle with a class (for example with `next-themes`), declare it in CSS (v4):

CSS

```
@import "tailwindcss";
@custom-variant dark (&:where(.dark, .dark *));
```

7.10 A complete component

TSX

```
export function PricingCard({ name, price, features }: { name: string; price: number; features: string[] }) {
  return (
    <div className="mx-auto w-full max-w-sm rounded-2xl border border-gray-200 bg-white p-6 shadow-sm
      transition hover:shadow-lg dark:border-gray-800 dark:bg-gray-900">
      <h3 className="text-sm font-semibold uppercase tracking-wide text-blue-600">{name}</h3>
      <p className="mt-2 flex items-baseline gap-1">
        <span className="text-4xl font-bold">${price}</span>
        <span className="text-gray-500">/month</span>
      </p>
      <ul className="mt-6 space-y-2 text-sm text-gray-600 dark:text-gray-300">
        {features.map((f) => (
          <li key={f} className="flex gap-2"><span aria-hidden>&#10003;</span>{f}</li>
        ))}
      </ul>
      <button className="mt-8 w-full rounded-lg bg-blue-600 py-2.5 font-medium text-white
        hover:bg-blue-700 focus-visible:outline-2 focus-visible:outline-offset-2
        focus-visible:outline-blue-600">
        Choose plan
      </button>
    </div>
  );
}
```

8. Tailwind CSS - Intermediate and Advanced

CATEGORY

Styling: scaling Tailwind. Theme customisation with `@theme`, component variants, `cn()` utility, `@apply`, plugins, animations, accessibility, performance and v3 to v4 migration. Level: Intermediate to Advanced.

8.1 Customising the theme with `@theme` (v4)

In v4 your design tokens are **CSS variables** declared in `@theme`; each namespace generates matching utilities automatically.

CSS

```
@import "tailwindcss";

@theme {
  /* colours: --color-* -> bg-brand-500, text-brand-500, border-brand-500 ... */
  --color-brand-50: oklch(0.97 0.02 250);
  --color-brand-500: oklch(0.62 0.19 255);
  --color-brand-700: oklch(0.45 0.18 258);
  /* fonts: --font-* -> font-display */
  --font-display: "Poppins", ui-sans-serif, system-ui, sans-serif;

  /* breakpoints: --breakpoint-* -> 3xl: variant */
  --breakpoint-3xl: 1920px;

  /* spacing base (all spacing utilities derive from it) */
  --spacing: 0.25rem;
  /* radii, shadows, easing, animations */
  --radius-card: 1rem; /* rounded-card */
  --shadow-soft: 0 4px 20px rgb(0 0 0 / 0.08); /* shadow-soft */
  --ease-snappy: cubic-bezier(0.2, 0, 0, 1); /* ease-snappy */
}
```

HTML

```
<button class="rounded-card bg-brand-500 font-display shadow-soft hover:bg-brand-700 3xl:text-xl">Go</button>
```

Namespace	Generates
<code>--color-*</code>	<code>bg-*</code> , <code>text-*</code> , <code>border-*</code> , <code>ring-*</code> , <code>fill-*</code> ...
<code>--font-*</code>	<code>font-*</code> families
<code>--text-*</code>	<code>text-*</code> sizes
<code>--breakpoint-*</code>	Responsive variants
<code>--spacing</code>	<code>p-*</code> , <code>m-*</code> , <code>gap-*</code> , <code>w-*</code> ... (multiples of the base)
<code>--radius-*</code>	<code>rounded-*</code>
<code>--shadow-*</code>	<code>shadow-*</code>
<code>--animate-*</code>	<code>animate-*</code>

CSS

```
/* Reset the default palette entirely and keep only your own tokens */
@theme { --color-*: initial; --color-ink: #111; --color-paper: #fff; --color-accent: #ff4d00; }

/* Map existing CSS variables (e.g. shadcn/ui, next-themes) without emitting new variables */
@theme inline { --color-background: var(--background); --color-foreground: var(--foreground); }

:root { --background: #fff; --foreground: #111; }
.dark { --background: #0b0b0c; --foreground: #f5f5f5; }
```

8.2 v3 vs v4 (migration cheat sheet)

Topic	Tailwind v3	Tailwind v4
Entry CSS	<code>@tailwind base; @tailwind components; @tailwind utilities;</code>	<code>@import "tailwindcss";</code>
Config	<code>tailwind.config.js (theme.extend)</code>	<code>@theme { ... }</code> in CSS (JS config optional via <code>@config</code>)
Content detection	<code>content: ["./src/**/*.tsx"]</code>	Automatic (uses <code>.gitignore</code>), <code>@source</code> to add paths
Dark mode class	<code>darkMode: "class"</code>	<code>@custom-variant dark (&:where(.dark, .dark *))</code>
Plugins	JS plugins	CSS <code>@plugin, @utility, @variant</code>
Engine	JS	Rust (Oxide): much faster full and incremental builds
Colours	sRGB hex palette	Modern P3 / <code>oklch</code> palette
CSS variables	Few	All tokens exposed as <code>var(--color-...)</code>
Container queries	Plugin	Built in (<code>@container, @md</code> : variants)
Browser support	Older browsers	Modern browsers (Safari 16.4+, Chrome 111+, Firefox 128+)
Some renamed utilities	<code>shadow-sm, rounded-sm, outline-none, bg-opacity-50</code>	<code>shadow-xs, rounded-xs, outline-hidden, bg-black/50</code>

Upgrade tool: `npm @tailwindcss/upgrade` (requires Node 20+).

8.3 Reusable components: the right way

Do not build a design system out of `@apply`. **Do** create framework components (React) and encode variants.

TSX

```
// lib/utls.ts - the standard helper: merge conditionally and resolve conflicts
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";
export const cn = (...inputs: ClassValue[]) => twMerge(clsx(inputs));

// cn("px-4 py-2", isBig && "px-8", className) -> later px-8 wins over px-4 (twMerge removes the conflict)
```

TSX

```
// components/ui/button.tsx - variants with class-variance-authority (cva)
import { cva, type VariantProps } from "class-variance-authority";
import { cn } from "@lib/utls";

const button = cva(
  "inline-flex items-center justify-center gap-2 rounded-md font-medium transition-colors " +
  "focus-visible:outline-2 focus-visible:outline-offset-2 disabled:pointer-events-none disabled:opacity-50",
  {
    variants: {
      variant: {
        primary: "bg-brand-500 text-white hover:bg-brand-700 focus-visible:outline-brand-500",
        outline: "border border-gray-300 bg-transparent hover:bg-gray-50",
        ghost: "hover:bg-gray-100",
        danger: "bg-red-600 text-white hover:bg-red-700",
      },
      size: { sm: "h-8 px-3 text-sm", md: "h-10 px-4", lg: "h-12 px-6 text-lg" },
    },
    compoundVariants: [{ variant: "outline", size: "lg", class: "border-2" }],
    defaultVariants: { variant: "primary", size: "md" },
  }
);

type Props = React.ComponentProps<"button"> & VariantProps<typeof button>;
```

```
export function Button({ className, variant, size, ...props }: Props) {
  return <button className={cn(button({ variant, size }), className)} {...props} />;
}

// <Button variant="outline" size="lg" className="w-full">Cancel</Button>
```

NOTE

Dynamic class names do not work. Tailwind finds classes by scanning source text, so `bg-${color}-500` generates nothing. Write complete class names (`{ red: "bg-red-500", blue: "bg-blue-500" }[color]`) or use CSS variables for truly dynamic values (`style={{ "--c": value }}` with `bg(--c)`).

8.4 @apply, @layer, @utility

CSS

```
@import "tailwindcss";

/* Base styles for plain elements (typography for CMS content, resets) */
@layer base {
  h1 { @apply text-3xl font-bold tracking-tight; }
  a { @apply text-brand-500 underline-offset-4 hover:underline; }
}

/* A rare, reasonable use of @apply: styling content you cannot add classes to (Markdown output) */
.prose-lite p { @apply my-4 leading-7; }

/* Custom utility (v4) - works with all variants: hover:scrollbar-hidden, md:scrollbar-hidden */
@utility scrollbar-hidden {
  scrollbar-width: none;
  &::-webkit-scrollbar { display: none; }
}

/* Custom variant (v4) */
@custom-variant hocus (&:is(:hover, :focus-visible));
```

Technique	Use when
Utilities inline	Default
Framework component (+ <code>cva</code>)	A repeated element (Button, Card, Badge)
<code>@apply</code>	Third-party/Markdown HTML, tiny base styles
<code>@utility</code>	New single-purpose utility you want variants for
<code>@theme</code> token	A design value used everywhere (colour, radius, font)

8.5 Arbitrary values and modern CSS features

HTML

```
<div class="top-[117px] w-[calc(100%-2rem)] bg-[#1dalf2] grid-cols-[1fr_2fr] text-[length:var(--size)]">
<div class="bg(--brand) text-(length:--fs)"> <!-- v4 variable shorthand -->
<div class="[mask-type:luminance] [&svg]:size-4 [&p]:mb-2"> <!-- arbitrary properties and child selectors -->
<div class="@container"><div class="grid grid-cols-1 @md:grid-cols-3"> <!-- container queries -->
<div class="text-balance wrap-anywhere"> <!-- text-wrap utilities -->
<div class="bg-linear-to-br from-indigo-500 via-purple-500 to-pink-500">
<div class="shadow-lg/20 inset-shadow-sm 3d-transform rotate-x-12"> <!-- v4 additions -->
```

8.6 Animation and transitions

HTML

```
<button class="transition duration-200 ease-out hover:-translate-y-0.5 hover:shadow-lg">Lift</button>
<div class="animate-spin size-5 rounded-full border-2 border-gray-300 border-t-blue-600"></div>
<div class="animate-pulse bg-gray-200 h-4 rounded"></div>      <!-- skeleton -->
<span class="animate-bounce">v</span>
<div class="motion-safe:animate-fade-in motion-reduce:animate-none">...</div>
```

CSS

```
@theme {
  --animate-fade-in: fade-in 0.4s ease-out both;
  @keyframes fade-in { from { opacity: 0; transform: translateY(8px); } to { opacity: 1; transform: none; } }
}
```

Starting-style (v4) supports enter animations for popovers and dialogs without JS: `starting:opacity-0`.

8.7 Accessibility with Tailwind

- Always provide **visible focus**: `focus-visible:outline-2 focus-visible:outline-offset-2` (never remove outlines without a replacement; `outline-none` in v3 became `outline-hidden` in v4).
- Hide visually but keep for screen readers: `sr-only`; undo with `not-sr-only`.
- Respect user preferences: `motion-reduce:`, `prefers-contrast` via `contrast-more:`, `forced-colors:`.
- Check **colour contrast** (4.5:1 for normal text). Light grays like `text-gray-400` on white fail.
- Use semantic elements; Tailwind does not make a `div` into a button.
- Minimum touch target about 44x44 px: `min-h-11 min-w-11`.

8.8 Component libraries on Tailwind

Library	Idea	Notes
shadcn/ui	Copy-in components built from Radix UI + Tailwind + <code>cva</code>	You own the code; most popular with Next.js
Headless UI / Radix UI / React Aria	Unstyled accessible primitives	Combine with your own Tailwind styles
DaisyUI	Component class names as a Tailwind plugin (<code>btn btn-primary</code>)	Fast, less custom
Flowbite, Preline	Ready-made component sets	Quick UI kits
Tailwind Plus (UI Blocks)	Official paid templates	Production-ready layouts

8.9 Performance and tooling

- Output CSS is usually **under 10-20 kB** compressed because only used utilities are generated.
- **Prettier plugin** `prettier-plugin-tailwindcss` sorts classes consistently. **ESLint** plugin flags conflicting or unknown classes.
- VS Code **Tailwind CSS IntelliSense** gives autocomplete, previews and lint inside class strings (including `cn()/cva()` via `classRegex` settings).
- Avoid generating huge arbitrary-value sets; keep tokens in `@theme`.
- Use `@source "../node_modules/some-ui-lib"` when classes come from a package that is not scanned by default.

8.10 Tailwind common mistakes

Mistake	Fix
Building class names dynamically (<code>`text-\${size}`</code>)	Map to full class strings
Using <code>@apply</code> for everything	Components + utilities
Desktop-first thinking (<code>lg:</code> for mobile fix-ups)	Mobile-first: base = mobile, add <code>md:/lg:</code>
Conflicting classes (<code>p-2 p-4</code>)	<code>tailwind-merge</code> via <code>cn()</code>
Missing focus styles	<code>focus-visible:</code> utilities
Overusing arbitrary values	Add a theme token
Forgetting <code>dark:</code> pairs	Define semantic colour tokens that change per theme
Using v3 syntax with v4 packages	Follow the v4 migration table

9. Deep Comparisons

CATEGORY

Decision chapter. Next.js against its rivals, Tailwind against other styling approaches, and the major full-stack combinations compared on cost, speed, scale and team fit. Level: All levels.

9.1 Next.js vs other React / full-stack frameworks

Criteria	Next.js	React + Vite (SPA)	Remix / React Router 7	Astro	Nuxt (Vue)	SvelteKit
Rendering	SSR, SSG, ISR, streaming, RSC	CSR only	SSR + progressive enhancement	Static + islands	SSR/SSG	SSR/SSG
Routing	File-based (App Router)	Library (React Router)	File or config based	File-based	File-based	File-based
Data loading	Server Components, "use cache"	TanStack Query / effects	Loaders and actions	Frontmatter fetch	useFetch	load functions
Mutations	Server Actions, Route Handlers	Call an API	Actions (forms)	Endpoints	Server routes	Form actions
SEO	Excellent	Weak by default	Excellent	Excellent	Excellent	Excellent
JS shipped	Moderate (RSC reduces it)	Highest	Moderate	Lowest	Moderate	Low
Learning curve	Medium-high (caching, RSC)	Low	Medium	Low	Medium	Low-Medium
Ecosystem and jobs	Largest	Large	Medium	Growing	Large	Growing
Hosting	Vercel best; Node/Docker anywhere	Any static host	Node/edge	Static hosts	Node/edge	Node/edge
Best for	SaaS, e-commerce, dashboards, content + app mixes	Internal tools, apps behind login	Form-heavy apps, web standards fans	Content sites, blogs, docs	Vue teams	Small bundles, simple DX

NOTE

Pick Next.js when you need SEO, performance, server rendering and a rich ecosystem and you already use React. **Pick plain React + Vite** when SEO does not matter (admin panels, internal tools) and you already have a separate backend (for example Spring Boot).

9.2 Tailwind CSS vs other styling approaches

Criteria	Tailwind CSS	Bootstrap	CSS Modules	styled-components / Emotion	Plain CSS / Sass
Philosophy	Utility classes	Pre-built components	Scoped CSS files	CSS-in-JS	Hand-written stylesheets
Customisation	Very high (tokens + utilities)	Medium (override variables)	Total	Total	Total
Look out of the box	Nothing (you design it)	Recognisable Bootstrap look	Nothing	Nothing	Nothing
Speed of building	Fast once learned	Very fast for standard UI	Slower	Medium	Slower

Criteria	Tailwind CSS	Bootstrap	CSS Modules	styled-components / Emotion	Plain CSS / Sass
Runtime cost	None	None	None	Yes (JS runtime; problematic with Server Components)	None
Works with React Server Components	Yes	Yes	Yes	Needs special setup	Yes
CSS size	Small (only used utilities)	Larger	Small	Small	Depends
Consistency	Enforced by design scale	High	Depends on discipline	Depends	Depends
Learning curve	Learn utility names	Learn components	Low	Medium	Low-High
Team scaling	Excellent with components	Good	OK	OK	Needs naming conventions (BEM)
Dark mode, responsive	Built-in variants	Built-in utilities	Manual	Manual	Manual
Best for	Custom designs, design systems	Prototypes, admin dashboards, quick standard UI	Teams who want real CSS	Dynamic theming in client apps	Simple sites

HTML

```

<!-- Same card in three approaches -->

<!-- Bootstrap -->
<div class="card shadow-sm"><div class="card-body"><h5 class="card-title">Title</h5>
  <a class="btn btn-primary">Go</a></div></div>

<!-- Tailwind -->
<div class="rounded-xl border p-4 shadow-sm"><h5 class="text-lg font-semibold">Title</h5>
  <a class="mt-3 inline-block rounded bg-blue-600 px-4 py-2 text-white">Go</a></div>

<!-- CSS Modules -->
<div class={styles.card}><h5 class={styles.title}>Title</h5><a class={styles.btn}>Go</a></div>

```

9.3 Full-stack architecture options compared

Architecture	Stack	Strengths	Weaknesses	Best for
A. Next.js full-stack monolith	Next.js + TypeScript + Tailwind + Prisma/Drizzle + Postgres	One repo, one language, fastest development, shared types, simple deploy	Backend logic tied to the web app; heavy compute and complex domains get awkward	Startups, MVPs, SaaS, content apps
B. Next.js frontend + Spring Boot API	Next.js (BFF) + Spring Boot + Postgres	Strong domain layer, transactions, security, long-term support; UI team and API team independent	Two languages and two deployments; contract management needed	Enterprise, fintech, existing Java teams
C. React SPA + Spring Boot API	React + Vite + TanStack Query + Spring Boot	Simple separation, easy to hire for, CDN hosting for UI	No SSR/SEO; auth and CORS need care	Internal tools, admin systems
D. Next.js + NestJS/Express API	Next.js + Node backend	One language (TypeScript) across everything, share Zod schemas/types	Node ecosystem churn; CPU-heavy work needs care	Product teams who are all TypeScript

Architecture	Stack	Strengths	Weaknesses	Best for
E. Microservices + BFF	Several Spring Boot/Node services + Next.js BFF + Kafka	Scale teams and services independently	High complexity and cost	Large organisations only

TEXT

- A. Browser <-> Next.js (UI + Server Actions + DB access) <-> Postgres
- B. Browser <-> Next.js (RSC + BFF) <-> Spring Boot (domain, security, batch) <-> Postgres, Kafka
- C. Browser (React SPA from CDN) ---REST/JSON---> Spring Boot <-> Postgres
- D. Browser <-> Next.js <-> NestJS API <-> Postgres (shared TypeScript types package)

9.4 The whole technology comparison on one page

Dimension	Spring Boot	Node.js / NestJS	Next.js	React (SPA)	Tailwind	TypeScript
Runs on	JVM	V8	Node (server) + browser	Browser	Build time only	Compile time only
Layer	Backend	Backend	Full-stack web	Frontend	Styling	Language
Typing	Static (Java)	Static with TS	Static with TS	Static with TS	n/a	Is the typing
Learning curve	Steep	Gentle to medium	Medium-high	Medium	Low-medium	Medium
Time to MVP	Medium	Fast	Fast	Fast	Fast	+small setup
Scalability	Excellent	Very good (horizontal)	Very good	n/a (static)	n/a	n/a
Enterprise adoption	Very high	High	High (growing)	Very high	High	Very high
Typical salary demand (market tendency)	High	High	High	High	Medium (skill, not role)	Expected skill
Biggest risk	Over-engineering, verbosity	Unstructured growth, dependency churn	Caching complexity, frequent changes	Choice overload	Messy class lists without components	Type gymnastics

9.5 Choosing by project type

Project	Recommended	Why
Portfolio / blog / marketing site	Next.js (SSG) or Astro + Tailwind	SEO, speed, simple hosting
SaaS MVP by a small team	Next.js + TypeScript + Tailwind + Postgres (+ Auth.js)	Fastest end to end
E-commerce	Next.js (ISR + "use cache") + headless commerce API	SEO and performance drive revenue
Admin dashboard / internal tool	React + Vite + TanStack Query + Tailwind + shadcn/ui, backend of your choice	No SEO need, rich interactivity
Banking / insurance / ERP backend	Spring Boot (+ Postgres, Kafka) with a React or Next.js UI	Transactions, security, longevity
Real-time collaboration, chat	Node.js (Socket.IO / WebSockets) + React	Event-driven model

Project	Recommended	Why
Public API for many clients	Spring Boot or NestJS with OpenAPI	Contracts, versioning, security
Mobile app + web	React Native/Expo + shared TypeScript packages + any API	Share logic and types
Data-heavy batch / ETL	Spring Boot (Spring Batch) or Python	JVM throughput
Enterprise portal modernisation	Next.js BFF in front of existing Spring Boot services	Gradual, low-risk migration

10. Capstone: Full-Stack Task Manager (Next.js + Tailwind + TypeScript + Spring Boot)

CATEGORY

Worked project. A complete vertical slice that combines all three volumes: the Spring Boot (or Node) Task API from Volume 1, TypeScript types and React concepts from Volume 2, and Next.js with Tailwind from this volume. Level: Intermediate to Advanced.

10.1 Architecture

TEXT

```
Browser
| HTML (RSC) + small client JS           /tasks, /tasks/new, /login
v
Next.js 16 (App Router) - Server Components fetch data, Server Actions mutate, proxy.ts guards routes
| REST + Bearer JWT (httpOnly cookie on the browser side only)
v
Spring Boot 3.5/4 (Task API from Vol 1, chapter 12) - validation, JPA, Spring Security
|
PostgreSQL
```

10.2 Folder layout

TEXT

```
web/
  app/
    layout.tsx  globals.css  page.tsx
    login/page.tsx  login/actions.ts
    tasks/
      page.tsx      <- Server Component: list + filters
      loading.tsx   error.tsx
      new/page.tsx  new/form.tsx
      actions.ts    <- createTask, completeTask, deleteTask
      task-row.tsx  <- Client Component (optimistic toggle)
  lib/
    session.ts  dal.ts  api.ts  schemas.ts
  proxy.ts
  next.config.ts  .env.local
```

10.3 Typed API client and schemas

TS

```
// lib/schemas.ts
import { z } from "zod";
export const Task = z.object({ id: z.number(), title: z.string(), priority: z.number(),
                             status: z.enum(["OPEN", "DONE"]) });
export type Task = z.infer<typeof Task>;
export const TaskPage = z.object({ content: z.array(Task), totalPages: z.number(), totalElements: z.number()
                                   });
export const CreateTask = z.object({ title: z.string().trim().min(1, "Title is required").max(100),
                                     priority: z.coerce.number().int().min(1).max(5) });

// lib/api.ts
import "server-only";
import { getSession } from "../session";

export async function api<T>(path: string, schema: z.ZodType<T> | null, init: RequestInit = {}): Promise<T> {
  const s = await getSession();
  const res = await fetch(`${process.env.API_URL}${path}`, {
    ...init,
    headers: { "Content-Type": "application/json", Authorization: `Bearer ${s?.token}`, ...init.headers },
  });
  if (!res.ok) throw new Error(`API ${res.status}`);
  return schema && res.status !== 204 ? schema.parse(await res.json()) : (undefined as T);
}
```

10.4 List page (Server Component) with Tailwind

TSX

```
// app/tasks/page.tsx
import Link from "next/link";
import { api } from "@lib/api";
import { TaskPage } from "@lib/schemas";
import { TaskRow } from "../task-row";

export const metadata = { title: "Tasks" };
export default async function TasksPage({ searchParams }: { searchParams: Promise<{ status?: string; page?: string }> }) {
  const { status, page = "0" } = await searchParams;
  const qs = new URLSearchParams({ page, size: "10", ...(status ? { status } : {}) });
  const data = await api(`/api/tasks?${qs}`, TaskPage);

  const tab = (value: string | undefined, label: string) => (
    <Link
      href={value ? `/tasks?status=${value}` : "/tasks"}
      className={`rounded-full px-3 py-1 text-sm ${status === value
        ? "bg-blue-600 text-white" : "bg-gray-100 text-gray-700 hover:bg-gray-200"}`}
    >{label}</Link>
  );

  return (
    <section className="space-y-4">
      <div className="flex items-center justify-between">
        <h1 className="text-2xl font-bold">Tasks <span
          className="text-gray-400">({data.totalElements})</span></h1>
        <Link href="/tasks/new" className="rounded-lg bg-blue-600 px-4 py-2 text-white hover:bg-blue-700">New
          task</Link>
      </div>

      <div className="flex gap-2">{tab(undefined, "All")}{tab("OPEN", "Open")}{tab("DONE", "Done")}</div>
      {data.content.length === 0 ? (
        <p className="rounded-lg border border-dashed p-8 text-center text-gray-500">No tasks yet.</p>
      ) : (
        <ul className="divide-y rounded-xl border bg-white shadow-sm">
          {data.content.map((t) => <TaskRow key={t.id} task={t} />)}
        </ul>
      )}
    </section>
  );
}
```

10.5 Server Actions

TS

```
// app/tasks/actions.ts
"use server";
import { revalidatePath } from "next/cache";
import { redirect } from "next/navigation";
import { api } from "@lib/api";
import { CreateTask, Task } from "@lib/schemas";

export type State = { errors?: Record<string, string[] | undefined>; message?: string } | undefined;
export async function createTask(_: State, form: FormData): Promise<State> {
  const parsed = CreateTask.safeParse(Object.fromEntries(form));
  if (!parsed.success) return { errors: parsed.error.flatten().fieldErrors };
  try {
    await api("/api/tasks", Task, { method: "POST", body: JSON.stringify(parsed.data) });
  } catch {
    return { message: "Could not create the task. Please try again." };
  }
  revalidatePath("/tasks");
  redirect("/tasks"); // outside try/catch on purpose
}

export async function completeTask(id: number) {
  await api(`/api/tasks/${id}/complete`, Task, { method: "PATCH" });
  revalidatePath("/tasks");
}

export async function deleteTask(id: number) {
  await api(`/api/tasks/${id}`, null, { method: "DELETE" });
  revalidatePath("/tasks");
}
```

10.6 Client Component with optimistic update

TSX

```
// app/tasks/task-row.tsx
"use client";
import { useOptimistic, useTransition } from "react";
import type { Task } from "@lib/schemas";
import { completeTask, deleteTask } from "../actions";

export function TaskRow({ task }: { task: Task }) {
  const [optimistic, setOptimistic] = useOptimistic(task, (t, status: Task["status"]) => ({ ...t, status }));
  const [pending, startTransition] = useTransition();
  const done = optimistic.status === "DONE";

  return (
    <li className="flex items-center gap-3 p-3">
      <button
        aria-label={done ? "Completed" : "Mark as done"}
        disabled={done || pending}
        onClick={() => startTransition(async () => { setOptimistic("DONE"); await completeTask(task.id); })}
        className={`size-5 rounded-full border-2 ${done ? "border-green-500 bg-green-500" : "border-gray-300 hover:border-blue-500"} `}
      />
      <span className={`flex-1 ${done ? "text-gray-400 line-through" : ""}`}>{optimistic.title}</span>
      <span className="rounded bg-gray-100 px-2 py-0.5 text-xs text-gray-600">P{task.priority}</span>
      <button
        onClick={() => startTransition(() => deleteTask(task.id))}
        aria-label={`Delete ${task.title}`}
        className="rounded p-1 text-gray-400 hover:bg-red-50 hover:text-red-600"
      >🗑️</button>
    </li>
  );
}
```

10.7 What the capstone proves

Concept	Where used
Server Components fetch data	<code>TasksPage</code> calls the Spring Boot API directly, ships no fetching JS
Client Components for interactivity	<code>TaskRow</code> handles clicks and optimistic UI
Server Actions	<code>createTask</code> , <code>completeTask</code> , <code>deleteTask</code> with Zod validation
Caching and invalidation	<code>revalidatePath("/tasks")</code> after each mutation
Streaming and error UI	<code>loading.tsx</code> , <code>error.tsx</code> in <code>/tasks</code>
TypeScript + Zod	One schema validates and types data (Vol 2)
Spring Boot API contract	Same endpoints and status codes as Vol 1, chapter 12
Tailwind	Utility classes for layout, states and responsive design
Security	JWT never exposed to JS (httpOnly cookie); authorisation also enforced by Spring

11. Decision Guide, Interview Q&A and Cheat Sheets

CATEGORY

Revision chapter. Fast answers for interviews, a one-page syntax cheat sheet per technology, and a master roadmap that combines all three volumes. Level: All levels.

11.1 Next.js interview questions

Question	Short answer
What is Next.js?	A React meta-framework adding routing, rendering strategies (SSR/SSG/ISR/streaming), server components/actions, optimisation and deployment tooling
App Router vs Pages Router?	App Router (<code>app/</code>) uses nested layouts, Server Components, streaming and Actions; Pages Router uses <code>getServerSideProps/getStaticProps</code> and is legacy
Server vs Client Components?	Server: run on server, async, no client JS, no hooks. Client (<code>"use client"</code>): interactive, hooks, browser APIs
What is hydration?	Attaching React event handling to server-rendered HTML in the browser
SSR vs SSG vs ISR?	HTML per request / at build time / at build time then refreshed on a schedule or on demand
What are Server Actions?	Async functions (<code>"use server"</code>) invoked from forms/components that run on the server; they are public endpoints needing auth and validation
Route Handler vs Server Action?	Handlers expose HTTP endpoints for any client; Actions are for your own UI mutations
What does <code>"use cache"</code> do?	Opt-in caching of a function, component or page (Cache Components, Next.js 16) with <code>cacheLife</code> and <code>cacheTag</code>
<code>revalidatePath</code> vs <code>revalidateTag</code> ?	By URL vs by label attached to cached data
What replaced <code>middleware.ts</code> ?	<code>proxy.ts</code> in Next.js 16
How do you protect routes?	Optimistic checks in <code>proxy.ts</code> plus real authorisation in the data layer / actions
Why are <code>params</code> awaited?	They are Promises since v15 to support streaming and partial rendering
How does <code>next/image</code> help?	Responsive sizes, modern formats, lazy loading, no layout shift
Deploy options?	Vercel, Docker (<code>output: "standalone"</code>), other clouds, static export
What is Turbopack?	Rust-based bundler, the default in Next.js 16 for dev and build

11.2 Tailwind interview questions

Question	Short answer
What is utility-first CSS?	Compose small single-purpose classes in markup instead of writing custom CSS rules
How does Tailwind keep CSS small?	It scans source files and generates only the classes used
How do you do responsive design?	Mobile-first with breakpoint prefixes (<code>md:</code> , <code>lg:</code>)
How do you customise?	v4: <code>@theme</code> in CSS; v3: <code>tailwind.config.js</code>

Question	Short answer
Dark mode?	<code>dark</code> : variant, system preference or class strategy via <code>@custom-variant</code>
<code>@apply</code> vs components?	Prefer framework components + utilities; <code>@apply</code> only in base or third-party content
Why don't dynamic class names work?	Tailwind detects complete strings in source; interpolated names are never generated
What is <code>group/peer</code> ?	Style children from parent state / siblings from an element's state
How do you merge conflicting classes?	<code>tailwind-merge</code> inside a <code>cn()</code> helper
What changed in v4?	CSS-first config, Rust engine, auto content detection, container queries, modern colours

11.3 Mixed "which one" questions

Question	Answer
Is Next.js a backend?	It can host backend code (Route Handlers, Server Actions), but it is a web framework; heavy domain logic fits Spring Boot or NestJS better
Do I need Node.js if I use Spring Boot?	Yes for the frontend toolchain (Next.js, Vite, npm), not for the Spring runtime
Is TypeScript required for React?	No, but it is the industry default
Tailwind or Bootstrap?	Tailwind for custom designs and design systems; Bootstrap for quick standard UI
React or Next.js?	React is the library; Next.js is a framework built on React. Use Next.js for SEO and server features, Vite + React for SPAs
Spring Boot or Node.js for a new API?	Spring Boot for complex domains and Java teams; Node/NestJS for TypeScript teams and I/O-heavy or real-time services
Can I use all of them together?	Yes: Spring Boot API + Next.js (React, TypeScript) + Tailwind is a very common enterprise stack

11.4 Syntax cheat sheet: Next.js

Need	Code
Page	<code>app/x/page.tsx</code> with <code>export default function Page() {}</code>
Dynamic route	<code>app/x/[id]/page.tsx</code> , <code>const { id } = await params</code>
Layout	<code>app/x/layout.tsx</code> receiving <code>children</code>
Client component	<code>"use client"</code> first line
Server Action	<code>"use server"</code> in file or function
Route Handler	<code>app/api/x/route.ts</code> , <code>export async function GET()</code>
Redirect / 404	<code>redirect("/login")</code> , <code>notFound()</code>
Revalidate	<code>revalidatePath("/x")</code> , <code>updateTag("tag")</code> , <code>revalidateTag("tag", "max")</code>
Cache	<code>"use cache"</code> + <code>cacheLife("hours")</code> + <code>cacheTag("x")</code>
Metadata	<code>export const metadata / generateMetadata</code>
Env vars	<code>process.env.X</code> (server), <code>NEXT_PUBLIC_X</code> (browser)
Image / font	<code>next/image</code> , <code>next/font/google</code>
Proxy	<code>proxy.ts</code> exporting <code>proxy(req)</code> and <code>config.matcher</code>

11.5 Syntax cheat sheet: Tailwind

Need	Classes
Centre a box	<code>flex items-center justify-center</code>
Container	<code>mx-auto max-w-5xl px-4</code>
Card	<code>rounded-xl border bg-white p-6 shadow-sm</code>
Primary button	<code>rounded-lg bg-blue-600 px-4 py-2 font-medium text-white hover:bg-blue-700</code>
Responsive grid	<code>grid grid-cols-1 gap-4 md:grid-cols-2 lg:grid-cols-3</code>
Sticky header	<code>sticky top-0 z-40 border-b bg-white/80 backdrop-blur</code>
Truncate text	<code>truncate</code> or <code>line-clamp-2</code>
Focus ring	<code>focus-visible:outline-2 focus-visible:outline-offset-2 focus-visible:outline-blue-600</code>
Dark mode	<code>bg-white dark:bg-gray-900</code>
Hide on mobile	<code>hidden md:block</code>
Skeleton	<code>animate-pulse rounded bg-gray-200</code>

11.6 Master roadmap across all three volumes

Phase	We eks	Backend (Vol 1)	Language and UI (Vol 2)	Framework and Style (Vol 3)	Deliverable
1. Foundations	1-2	Java or Node basics, HTTP and REST	JS refresher, TypeScript basics, React basics	HTML/CSS refresher, Tailwind basics	Static pages + small React app
2. Core	3-5	CRUD API, JPA/Prisma, validation	Hooks, forms, TanStack Query, routing	Next.js routing, layouts, Server Components	Task API + UI
3. Production skills	6-8	Security, testing, Docker	Testing Library, performance	Server Actions, caching, auth, deployment	Full-stack app deployed
4. Advanced	9-12	Kafka, caching, observability, virtual threads	Patterns, RSC, compiler, advanced TypeScript	Parallel routes, BFF, CI/CD, performance tuning	Portfolio capstone
5. Interview prep	13-14	Chapter Q&A in Vol 1	Chapter Q&A in Vol 2	Chapter Q&A in Vol 3	Mock interviews

PRO TIP

Study method for deep preparation: (1) read the concept and category box, (2) type every example yourself, (3) answer the Q&A table out loud without looking, (4) rebuild the capstone with a variation (different entity, different backend), (5) explain the comparison tables in your own words - if you can say *why* one is better for a scenario, you have mastered the topic.

11.7 Final one-page summary

- **Spring Boot** = batteries-included Java backend: DI, JPA, Spring Security, Actuator. Strongest for complex, long-lived, transaction-heavy systems.
- **Node.js** = JavaScript runtime with an event loop: Express/Fastify/NestJS. Strongest for I/O-heavy APIs, real-time and TypeScript-everywhere teams.
- **TypeScript** = typed JavaScript. Use `strict`, validate external data with Zod, model state with discriminated unions.
- **React** = declarative components, hooks, one-way data flow. Keep state minimal, colocate it, use TanStack Query for server data.

- **Next.js** = React plus server rendering, routing, caching, Server Actions. Default to Server Components, add `"use client"` only where needed, cache explicitly.
- **Tailwind CSS** = utility-first styling with design tokens in `@theme`. Extract components not classes; mobile first; keep accessibility in mind.
- **Together:** Spring Boot (core domain) + Next.js with React and TypeScript (UI/BFF) + Tailwind (styling) is a proven, scalable and hireable stack.